

Why the $\forall$ PO-Free Fragment Is Decidable

A Guide for the Tableau-Minded

Michael Wessel — with Claude (Opus 4.8, Anthropic)

2026-07-22

A companion to the $\mathcal{ALCI}_{\text{RCC5}}$ decidability project, pitched one level more technical than `WHY_ITS_HARD.md`: for a reader who knows basic description logics, tableau calculi, and blocking, but not automata theory or model theory. The goal is not to walk you through the proofs — it is to put enough intuition and enough hard, checkable facts in front of you that you can honestly say “yes, I believe that.”

Honesty note, up front: the fragment theorem is **theorem-level, not end-to-end machine-certified**. Its soundness crux is kernel-checked in Lean 4 (`formal/POFreeLift.lean`, zero sorries), its algebraic ingredients are exhaustively machine-checked, and two independent argument routes arrive at it — but the full assembly has not been formalized, and the project’s standing label applies: *strongly supported, not certified*.

1 The claim

Theorem ($\forall$ PO-free fragment). Concept satisfiability for the $\forall$ PO-free fragment of $\mathcal{ALCI}_{\text{RCC5}}$ — concepts in NNF containing **no subformula of the form** $\forall \text{PO}.D$ — is decidable, with a computable live-width bound $K(C_0) = (1 + m)^2 2^{2^m}$, where $m = |\text{cl}(C_0)|$.

Read the fine print, because it is generous. The fragment **keeps**:

- $\exists \text{PO}.D$ — “overlaps something satisfying D ”;
- $\forall \text{DR}.D$ and $\exists \text{DR}.D$ — universal *and* existential disjointness;
- all vertical modalities — $\exists \text{PP}$, $\forall \text{PP}$, $\exists \text{PPI}$, $\forall \text{PPI}$, nested to any depth;
- infinite models — $\exists \text{PP}.\top \sqcap \forall \text{PP}.\exists \text{PP}.\top$ is in the fragment and is satisfiable only in models with an infinite ascending PP-chain.

The *only* thing removed is the universal restriction over partial overlap. That is a strikingly thin excision, and the whole story of this document is why the difficulty of the full logic is concentrated in exactly that one constructor.

A calibration point that matters: the fragment is **not** a toy that dodges the hard diagnostics. The concept

$$C_{\text{force}} = \exists \text{PP} . (\exists \text{DR} . A) \sqcap \forall \text{DR} . \neg A$$

is $\forall\text{PO}$ -free and UNSAT — because $x\text{PP}y$ and $y\text{DR}z$ force $x\text{DR}z$ ($\text{comp}(\text{PP}, \text{DR}) = \{\text{DR}\}$), and then $\forall\text{DR}.\neg A$ kills the required $z:A$. Every naive tableau that ignores forced edges accepts it wrongly. So a decision procedure for the fragment must still do genuine composition-table reasoning; it just never has to fight the one battle that has defeated every full-logic attempt.

2 The setting, and why your tableau instincts don’t directly apply

Three facts about $\mathcal{ALCI}_{\text{RCC5}}$ that break the standard playbook. (Strong-EQ semantics throughout: EQ means identity, so between distinct elements only DR, PO, PP, PPI occur.)

Fact 1: every model is a complete graph. The five RCC5 relations are jointly exhaustive and pairwise disjoint. Any two elements of any model stand in exactly one of them — there is no “unrelated.” In $\mathcal{ALC}$, a tableau builds a *tree* and elements in different branches simply have nothing to do with each other. Here, every pair of nodes you ever create carries a relation, whether you asked for it or not, and the composition table

$$\rho(x, z) \in \text{comp}(\rho(x, y), \rho(y, z)) \quad \text{for all } x, y, z$$

constrains all of them, globally. Adding one node adds n new edges, each of which must close every triangle it participates in.

Fact 2: no finite model property. $\exists\text{PP}.\top \sqcap \forall\text{PP}.\exists\text{PP}.\top$ forces an infinite strictly ascending chain of regions (PP is a strict order under strong EQ — a PP-cycle would collapse by composition into $x\text{PP}x$, which is impossible). So you cannot hope to enumerate finite models: satisfiable concepts may have none.

Fact 3: local blocking is unsound or incomplete. In $\mathcal{ALC}$, blocking works because “same label set” means “same future obligations.” Here a node’s future obligations depend on its relations to *the whole graph built so far* (Fact 1), which no fixed-arity local summary captures. The project tried subset blocking, triangle blocking, quadruple types, profile blocking, double blocking — each fails on a concrete diagnostic. This is not a technicality; it is the open problem of the full logic (the width bound F6) in disguise.

So the fragment result has to earn its termination some other way. That way is the *finite-certificate architecture*, and it is worth understanding on its own before either proof route, because both routes are instances of it.

3 The architecture: finite certificates for infinite models

When models are legitimately infinite, “decidable” cannot mean “search for a model.” It means:

Replace “model” by “finite blueprint of a model,” so that (a) a computer can enumerate and check blueprints, (b) every blueprint that checks out really describes a model, and (c) every satisfiable concept has a blueprint of computably bounded size.

Call the blueprint a **certificate**. A decidability proof in this architecture always has exactly three parts:

1. **A finite certificate format** with finitely many *finitely checkable* conditions (type coherence, witness presence, triangle closure on the finite object, ...). Checking a candidate certificate is trivially decidable.
2. **Soundness (unfolding)**. If a certificate passes the checks, the possibly-infinite structure it *generates* — obtained by unrolling its loops — is a genuine model of the concept. This is the direction that must not cheat: the checks are finite, the generated object is infinite, and you must prove the finite checks control the infinite unfolding.
3. **Completeness (extraction + compression)**. If the concept has *any* model, you can extract from it a certificate within the computable size bound. This is where “run out of colors” pigeonhole arguments live — and where a bound on the certificate’s size must come from.

Given all three, the decision procedure is embarrassing: enumerate all certificates up to the bound, check each, answer SAT iff one passes. All the mathematics is in (2) and (3); the algorithm itself is a for-loop.

Four remarks that make this concrete for a tableau reader.

A blocked tableau *is* a certificate. When your *ALC* tableau stops because node y is blocked by ancestor x , the finished tree with its blocking pointers is precisely a finite certificate: the “model” it describes is the infinite unraveling in which y ’s subtree “continues like x ’s.” Soundness of blocking = the unfolding lemma. So the architecture is not exotic; it is what blocking always secretly was.

Loops are generators, not quotients. One subtlety is promoted to a design principle here. The blocking loop must be read as “*the pattern continues like this*” (unfold the loop into infinitely many distinct copies), never as “*y equals x*” (bend the edge back). Under strong EQ the second reading is inconsistent for vertical loops — it would make a region a proper part of itself. The certificate has a PP-self-loop; the model it generates has an infinite chain of *distinct* elements. Finite generator, infinite unfolding, no finite quotient.

Under abstract semantics, model-checking is triangle-checking. A gift of the composition-table semantics: an interpretation is *any* set with a total labelling ρ satisfying (i) $\rho(d, d) = \text{EQ}$ and strong EQ, (ii) converse coherence, (iii) composition closure. Nothing else. So “the unfolding is a model” reduces to “every triangle of the unfolding is closed under the table” — a purely combinatorial statement, exactly the kind of thing one can prove by case analysis on where the three corners sit, and exactly what is kernel-checked in `P0FreeLift.lean`. (For networks that are only partially specified, RCC5’s *patchwork property* — path-consistent atomic networks are globally consistent, Renz–Nebel 1999 — completes the missing edges; for a fully labelled unfolding the network *is* the model outright.)

Live width: what has to be bounded. The size bound in (3) is the whole game. The key observation is that most of a certificate’s edges cost nothing: an edge whose value is *forced* by

composition from other edges (a “shadow”) need not be stored or chosen — it is recoverable. What must be bounded is the number of **live** edges: genuinely free choices attached to each part of the certificate. The conjecture that every satisfiable $\mathcal{ALCI}_{\text{RCC5}}$ concept admits a presentation of computably bounded live width is **F6** — the single open keystone of the full problem, machine-formalized as the hypothesis of a certified conditional theorem. **The $\forall\text{PO}$ -free fragment is precisely the place where a live width bound is actually proved** rather than conjectured. That is what “a fragment result” means in this project: F6, inhabited.

With the architecture fixed, both proofs of the fragment theorem can be stated in one sentence each:

- **Route 1 (two-tier quotient):** vertical blocking gives a finite set of chain-descriptors; the composition table makes every witness’s relation to a chain *stabilize*, so one certificate edge can honestly summarize infinitely many model edges — for DR, PP, PPI always, and for PO exactly when no $\forall\text{PO}$ constraint watches.
- **Route 2 (ordered-disjoint normal form):** an RCC5 model *is* just a strict order plus a well-behaved disjointness relation, with PO as the unconstrained residual; $\forall\text{PP}/\forall\text{PPI}/\forall\text{DR}$ obligations are order- and disjointness-closure conditions that propagate deterministically, and no propagation ever manufactures a $\forall\text{PO}$ obligation — so a standard tree tableau with blocking terminates.

The rest of this document unpacks those two sentences.

4 The algebraic heart: four forced cells, and the relation that is never forced

Everything in both routes reduces to a handful of cells of the RCC5 composition table. You can check each mentally with blobs, and each is machine-verified from finite set semantics (and, for those used in the Lean artifacts, kernel-checked).

The four singleton cells among proper relations:

$$\begin{aligned} \text{comp}(\text{PP}, \text{PP}) &= \{\text{PP}\} && \text{nesting is transitive} \\ \text{comp}(\text{PPI}, \text{PPI}) &= \{\text{PPI}\} && \dots \text{ in both directions} \\ \text{comp}(\text{PP}, \text{DR}) &= \{\text{DR}\} && \text{a part of something disjoint from } e \text{ is disjoint from } e \\ \text{comp}(\text{DR}, \text{PPI}) &= \{\text{DR}\} && \dots \text{ converse form} \end{aligned}$$

The third one deserves a picture: if the table is far from the moon, the coin *on* the table is far from the moon. Disjointness rains **down** into parts, deterministically. Together with transitivity of nesting, these four cells are the *only* deterministic machinery the whole algebra offers between distinct proper relations.

The two cells that define the problem:

$$\text{comp}(\text{PP}, \text{PO}) = \{\text{PP}, \text{PO}, \text{DR}\}, \quad \text{comp}(\text{PPI}, \text{PO}) = \{\text{PPI}, \text{PO}\} \quad \text{— not singletons.}$$

Picture the first: a cake y partially overlaps a plate e . Now take a crumb x of the cake ($x \text{ PP } y$). Where is the crumb relative to the plate? Maybe on the plate (PP), maybe straddling its rim (PO),

maybe on the tablecloth (DR). *Partial overlap tells you nothing determinate about parts.* The second cell is the upward version: if something inside y overlaps e , then y overlaps e or contains it — two options, not one.

And one global fact you can scan the whole table for:

No cell of the table equals $\{PO\}$. Partial overlap is never the uniquely forced outcome of any single composition. DR is forced by two cells, PP and PPI by transitivity — but no *single* path $x-y-z$ ever pins $x PO z$. PO is forced by nothing.

And the dual: PO is available wherever anything is. Scan the table again: every *non-singleton* proper cell **contains** PO — and this propagates along paths (of the ten composition-sets reachable by composing along paths of any length, every non-singleton one contains PO; machine-checked, wp26). Moreover every entry of the PO row and column contains PO, and $\text{comp}(PO, PO)$ is everything — so PO, once present, stays available in every direction, and an all-PO region constrains nothing. Put the two facts together and every pair of elements faces a clean dichotomy: either some composition path *pins* it (a singleton — forced, deterministic, a recoverable “shadow”), or **PO is among its options**. PO is the algebra’s escape valve: never forced on you, always offered where anything is. The only way a logic can close that valve is to make PO-edges *illegal* — and the only constructor that can do so is $\forall PO$. That, in one sentence, is why its removal is worth a decidability theorem: in the fragment, the escape valve is structurally un-closable, and the unfolding’s unboundedly many unpinned pairs all take the value that is always permitted and never watched.

Calibration, learned the hard way. That last fact is about *single* compositions. The *intersection* of several path constraints CAN force a PO edge — e.g. $\text{comp}(DR, PO) \cap \text{comp}(PPI, PO) = \{DR, PO, PP\} \cap \{PPI, PO\} = \{PO\}$. An earlier version of the project’s prose overstated “PO is never forced,” and a cold review caught it (the 16th, with a concrete two-path witness). Neither proof route below relies on the overstated version: both enforce *full* composition closure on every triangle, which subsumes multi-path forcing. Where “PO is loose” is used below, it is always in the precise single-cell sense above.

Yes, PO can be forced — and it is harmless. Do not read the caveat as fragility. Even inside the fragment, PO edges can be pinned: algebraically by joint paths (above), and logically by the *allowed* universals clashing out the alternatives. In

$$C = \forall PP. \neg A \sqcap \forall DR. \neg A \sqcap \exists PP. (\neg A \sqcap \exists PO. A)$$

the demanded $e : A$ sits at $\text{comp}(PP, PO) = \{PP, PO, DR\}$ from the root; $\forall PP. \neg A$ strikes PP, $\forall DR. \neg A$ strikes DR, and $\rho(\text{root}, e) = PO$ is *forced* (and the concept is satisfiable — $x = \{1, 2\}$, $y = \{1, 2, 3\}$, $e = \{2, 3, 4\}$). So the fragment can steer edges *into* PO. Why is that safe, when steering was the full logic’s downfall? Because a forced PO edge is **doubly inert**. *Logically inert*: the only constructor that fires on a PO edge — at either endpoint; PO is self-converse and the safety test is two-sided — is $\forall PO$, and none exists in the input or in any label that propagation will ever write (§6). The forced edge lands, and nothing happens. *Compositionally, it never forces onward alone*: PO is an argument of **no** singleton cell (look at the four: PP; PP, PP; DR, PPI; PPI, DR; PPI — PO appears in none), so a PO edge never single-handedly pins another edge;

it can narrow options in joint configurations, and full triangle closure tracks exactly that. Contrast forcing DR: a forced DR edge can fire a waiting $\forall\text{DR}$ at its far endpoint — that is literally C_{force} — real consequences, but *deterministic* ones the singleton cells propagate. Forcing PO has no consequences at all. Steering into the watched relations is trackable; steering into the unwatched one is a dead end for any adversarial concept.

Now the punchline of the whole section. A universal restriction $\forall R.D$ is a *standing obligation*: it must hold of every R -neighbor, including R -neighbors that arise *indirectly*, forced by composition, at any distance, at any time. Whether such an obligation is manageable depends entirely on whether the algebra moves R -neighbors around predictably:

- $\forall\text{DR}.D$ is **manageable** because DR propagates deterministically downward ($\text{comp}(\text{PP}, \text{DR}) = \{\text{DR}\}$): the set of DR-neighbors of a region grows in a controlled, forced, *monotone* way as you descend the nesting order. You always know where the obligation will land next.
- $\forall\text{PP}.D$ and $\forall\text{PPI}.D$ are **manageable** for the same reason via transitivity.
- $\forall\text{PO}.D$ is **the wild one**: PO-neighborhoods are not transported determinately in *either* vertical direction (the two non-singleton cells), and PO is never created by any single composition — so the obligation must be re-verified against a set of neighbors that the algebra refuses to pin down. The recurring failure shape across this project’s seventeen-review history — jointly coordinating horizontal edges against universal obligations at a gluing step — is only *hard* for edges no singleton cell controls; and PO is the relation with no singleton cell anywhere.

The fragment theorem is the statement that once $\forall\text{PO}$ is gone, the remaining obligations are exactly the ones the four singleton cells can carry. Two independent routes cash this in.

5 Route 1: the two-tier quotient (the chain-and-phase argument)

(Sources, under `papers/`: `two_tier_quotient_ALCIRCC5.tex` — Claude, April 2026, four revisions under GPT-5.4 review — and the fragment specialization `po_free_fragment_ALCIRCC5.tex`. Soundness crux certified 2026-07-18 in `formal/POFreeLift.lean`.)

5.1 The certificate

Since models contain infinite PP-chains, start where the infinity is. Walk up an infinite chain $d_0 \text{ PP } d_1 \text{ PP } d_2 \dots$ and record each element’s Hintikka type. There are at most 2^m types, so the sequence of types is eventually periodic — familiar pigeonhole, exactly the intuition behind blocking. Call the recurring cyclic pattern of types $(\tau_1, \dots, \tau_p)$ a **period descriptor**. There are at most 2^{2^m} descriptors.

The certificate (“two-tier quotient”) is:

- **Tier 1**: finitely many period descriptors, one **kernel** node per descriptor carrying its cyclic type pattern with a PP-self-loop (a generator: unfolds to the infinite chain, per §3);

- **Tier 2:** finitely many **designated witnesses** — at most one per existential demand per descriptor — with **one constant relation** recorded between each witness and each kernel: a single edge “kernel $\leftrightarrow e : R$ ”.

Plus finite checks: types are Hintikka, every $\exists$ demand has its witness, every recorded triangle is composition-closed, every recorded edge respects the universals in both endpoint types (the Safe test).

5.2 The crux: when can one edge summarize infinitely many?

Here is the exact point where this differs from $\mathcal{ALC}$ blocking, and where the reader should slow down. When the kernel unfolds into the infinite chain $d_0 \text{ PP } d_1 \text{ PP } \dots$, the certificate’s single edge “kernel $\leftrightarrow e : R$ ” must stand for *infinitely many* model edges: $\rho(d_i, e)$ for every i . The certificate records one value; the model needs a value at every rung. For the summary to be honest, the witness’s relation to the chain must be **the same at every rung that matters** — otherwise a finite certificate with constant interfaces simply cannot express the model.

Does the algebra cooperate? Watch a fixed external region e against a growing chain $d_0 \subset d_1 \subset d_2 \subset \dots$, using $\text{comp}(\text{PPI}, \cdot)$ to step up the chain:

from DR : next $\in \{\text{DR}, \text{PO}, \text{PPI}\}$
 from PO : next $\in \{\text{PO}, \text{PPI}\}$
 from PPI : next $\in \{\text{PPI}\}$ (absorbing)
 from PP : next $\in \{\text{PP}, \text{PO}, \text{PPI}\}$

Read it as a picture: the chain elements grow; a fixed e can go from disjoint, to overlapping, to *swallowed* (PPI) — and never backward. The relation **stabilizes**: after finitely many monotone switches it is constant forever. This is the “external-relation stabilization” lemma, and it is checkable from the table by eye.

Stabilization alone is not enough — the certificate needs the relation to be right at *all* phases, not just eventually. This is where the singleton cells become load-bearing, one relation at a time:

- DR — **backward forcing**. Suppose an $\exists\text{DR}$ -demand is satisfied by witness e at a late chain position d_i . Then for every earlier $j < i$: $d_j \text{ PP } d_i$ and $d_i \text{ DR } e$ give $\rho(d_j, e) \in \text{comp}(\text{PP}, \text{DR}) = \{\text{DR}\}$. **Forced**. A DR-witness picked late enough (past the point where all recurrent types have appeared) is automatically a DR-witness at every phase. The constant edge “kernel $\leftrightarrow e : \text{DR}$ ” is honest, for free.
- PP — **same cell, same argument**. If e contains d_i , then by $\text{comp}(\text{PP}, \text{PP}) = \{\text{PP}\}$ it contains every d_j below. Constant edge honest.
- PPI — **forward absorption**. If e is inside d_i , it is inside every d_j above ($\text{comp}(\text{PPI}, \text{PPI}) = \{\text{PPI}\}$). Constant edge honest.

So for DR, PP, PPI the composition table itself *converts* a one-position witness into an all-position witness. The finite summary is not an approximation; it is exact. If you believe those three singleton cells — and you can check them with blobs in a margin — you believe the constant-interface certificate is sound and complete for these three relations. That is the “yes!” moment for Route 1.

5.3 Where PO breaks it — a concrete satisfiable villain

For PO, *neither* direction is forced: $\text{comp}(\text{PP}, \text{PO}) = \{\text{PP}, \text{PO}, \text{DR}\}$ says earlier positions are free; $\text{comp}(\text{PPI}, \text{PO}) = \{\text{PPI}, \text{PO}\}$ says later ones are. A PO-witness at one phase need not be a PO-witness at any other. And this is not a hypothetical weakness of the proof — the two-tier paper exhibits a satisfiable concept that *provably defeats* the constant-interface architecture.

Take a chain with alternating types, τ_A at even positions demanding $\exists \text{PO}.A$, τ_B at odd positions demanding $\forall \text{PO}.\neg A$. Satisfiable? Yes — by **wandering witnesses**. For each k , witness w_k (satisfying A) is:

DR	to the chain positions below $2k$	(disjoint from the small ones)
PO	at exactly position $2k$	(overlaps that one)
PPI	to the positions above $2k$	(swallowed by the big ones)

Each w_k slides through the chain: disjoint, then transiently overlapping *exactly one* rung, then swallowed forever. Every even rung gets its $\exists \text{PO}.A$ witness; every odd rung has *no* PO-neighbor at all, so $\forall \text{PO}.\neg A$ holds vacuously; all the transitions are exactly the monotone ones from the table above; and the witnesses are pairwise DR by forced composition. A perfectly good infinite model.

But no *constant-interface* certificate describes it: a witness with recorded edge “kernel $\leftrightarrow w : \text{PO}$ ” would be PO to the τ_B -rungs too, violating $\forall \text{PO}.\neg A$. The demand at every phase is real, but no *single* witness serves two phases — the model needs infinitely many transient witnesses, and the finite summary has no slot for “transient.” The architecture is provably incomplete here. (Contrast DR: the analogous villain with $\exists \text{DR}.A / \forall \text{DR}.\neg A$ is *unrealizable* — backward forcing drags any late DR-witness back across the $\forall \text{DR}.\neg A$ rungs and clashes. The gap is a PO-only phenomenon; the table itself closes it for DR.)

5.4 Why $\forall \text{PO}$ -freeness dissolves the gap

Look at what actually went wrong: the constant-PO edge was fine *algebraically* (a constant-PO interface closes every triangle along the chain — check: $\text{PO} \in \text{comp}(\text{PP}, \text{PO})$ and $\text{PO} \in \text{comp}(\text{PPI}, \text{PO})$, so the constant value is always among the permitted ones) and failed only *logically* — it collided with a $\forall \text{PO}$ obligation at another phase. The Safe test for a PO-edge between types τ and σ has exactly two parts: the $\forall \text{PO}$ -universal part (“every $\forall \text{PO}.E \in \tau$ must have $E \in \sigma$,” and symmetrically), and propositional bookkeeping.

In a $\forall \text{PO}$ -free concept, no type contains any $\forall \text{PO}.E$. The $\forall \text{PO}$ -universal part of every Safe test is vacuous — at every phase, for every witness. A witness type containing D is PO-safe for *all* phases the moment it is PO-safe for one (“automatic PO-coherence,” Lemma 2 of the fragment note). The one relation the table refuses to transport is also the one relation nobody is watching. Constant-PO edges are algebraically consistent and logically unchallenged; the wandering villain needs a $\forall \text{PO}$ to exist, and there are none.

5.5 Counting, and the finish line

- Descriptors: $\leq 2^{2^m}$. Witnesses: at most one per demand per descriptor, $\leq (1 + m)$ each. Quotient size $Q \leq (1 + m) 2^{2^m}$; live width $K(C_0) \leq Q(1 + m) = (1 + m)^2 2^{2^m}$. Computable.

(Nobody claims it is tight — only computability matters, and no complexity bound is asserted.)

- **Completeness:** extract from any model — record recurrent types along each chain as descriptors, pick witnesses past the exhaustion index, let stabilization + backward forcing + forward absorption + automatic PO-coherence make them all-phase. Everything repeated folds into the kernels.
- **Soundness:** unfold each kernel into its infinite chain under the constant interface, and prove the unfolding is still composition-closed on *every* triple — including triples whose closure jointly forces a PO edge (the multi-path caveat of §4 is enforced, not assumed away). This **chain-unfolding lift** is the soundness crux, and it is the part that is now **kernel-checked**: `POFreeLift.lean` proves, in Lean 4 with zero sorries, that if the finite augmented network is composition-consistent and strong-EQ, then replacing the kernel by an infinite PP-chain under the constant interface preserves composition-consistency. And under abstract semantics a composition-closed total network *is* a model (§3) — so a valid certificate delivers an actual infinite model outright.

Enumerate certificates up to $K(C_0)$, check, done.

6 Route 2: the ordered-disjoint normal form (the structural argument)

(The regular-cover pivot, GPT-5.5, July 2026: `papers/rcc5_local_amalgamation_theory.tex`, probes wp47–wp49, wp82/wp83; normal form certified in `formal/RCC5NormalForm.lean`, both directions.)

Route 1 fought the composition table cell by cell. Route 2 wins by changing the data structure so the fight never starts.

6.1 A model is just an order and a disjointness relation

Certified normal form. A strong-EQ atomic RCC5 network is composition-closed **iff** it is an *ordered-disjoint structure*:

- $<$ (that is, PP) is a strict partial order;
- $\#$ (that is, DR) is symmetric, irreflexive, and **downward closed**: if $x\#y$, $x' \leq x$, $y' \leq y$, then $x'\#y'$;
- comparable pairs are never disjoint;
- **PO is the residual** — it is *defined* as “neither comparable nor disjoint.” It is not data. It is what’s left.

(Forward direction kernel-checked for arbitrary domains in `RCC5NormalForm.lean`; converse certified via an explicit canonical set representation, cross-checked exhaustively to size four and beyond, wp47/wp88.)

Let that reframing land: sixteen composition-table cells collapse into two familiar closure laws — transitivity of an order, downward closure of a disjointness — plus one compatibility condition. To build a model you never “place” a PO edge, ever. You build an order, you build a disjointness relation, you check two closure laws, and every pair you said nothing about *is* PO, automatically, consistently. The wild relation of §4 turns out to be bookkeeping for “no constraint here.”

With it come two machine-checked amalgamation facts we’ll need:

- **Free amalgamation** (wp48): two ordered-disjoint structures glued along a shared part, with no new order or disjointness across, form an ordered-disjoint structure. Cross pairs land on the residual — PO.
- **All-cross PO is always safe** (wp82): declaring *every* cross pair of two closed pockets PO is always composition-closed.

6.2 The tableau, and why it terminates

Now think like a tableau designer. Your nodes will form the vertical skeleton (a tree of $<$ -edges, one tree per component — the split-forest discipline); your constraints are $(<, \#)$; PO is whatever remains. Which obligations can arise, and how do they move?

Universals over PP, PPI, DR are exactly constraints on $<$ and $\#$: “everything below me satisfies D ,” “everything above me satisfies D ,” “everything disjoint from me satisfies D .” How do they propagate when edges compose? The general rule: if x carries $\forall R.D$ and $x \xrightarrow{S} y$, then y must carry $\forall T.D$ whenever $\text{comp}(S, T) \subseteq \{R\}$ (any T -successor of y is then a forced R -successor of x). Instantiating against the table yields the *complete* rule set (probe wp49, machine-checked from set semantics):

$\forall PP.D$	across PP:	require D , $\forall PP.D$	(transitivity of $<$)
$\forall PPI.D$	across PPI:	require D , $\forall PPI.D$	(transitivity, downward)
$\forall DR.D$	across PP:	require $\forall DR.D$	(my container’s disjointnesses rain down onto me — so my container inherits my duty)
$\forall DR.D$	across DR:	require D , $\forall PPI.D$	(my DR-neighbor’s parts are also my DR-neighbors)

— four rules, all riding the four singleton cells, and, decisively:

No propagation rule from a $\forall PP$, $\forall PPI$, or $\forall DR$ obligation ever produces a $\forall PO$ obligation. (Checked: no $\text{comp}(S, PO)$ is contained in $\{PP\}$, $\{PPI\}$, or $\{DR\}$.)

So the obligation vocabulary $\{\forall PP, \forall PPI, \forall DR\} \times \text{cl}(C_0)$ is **closed under propagation**. Start $\forall PO$ -free, stay $\forall PO$ -free — not just syntactically in the input, but dynamically, in every label the tableau will ever write. Obligations are drawn from a fixed finite set, and they move *deterministically* along the singleton cells (the same four cells as Route 1, wearing different clothes: transitivity of $<$, downward closure of $\#$). Node labels = type + obligation set = finitely many possibilities $\Rightarrow$ standard subformula blocking on the vertical tree terminates, with the loops read as generators (§3).

What about the demands?

- $\exists PP / \exists PPI / \exists DR$: create the witness in the skeleton or as a sibling; its forced consequences are computed by the two closure laws — finitely, deterministically (down-closure of $\#$ is a forced flood, but a flood of *shadows*: recoverable, costless, per §3).
- $\exists PO.D$: here is the beautiful step. Build a satisfying structure for D separately (recursion), then glue it on declaring **no cross order and no cross disjointness at all** — free amalgamation. Every cross pair, *including the demanding node and its witness*, lands on the residual: PO. The demand is satisfied by the default itself. Is that legal? *Algebraically* always (all-cross PO is always safe, wp82). *Logically* — a cross-PO edge could only violate a $\forall PO$ obligation, **and there are none, and there never will be any** (closure of the vocabulary). Both halves of “safe” hold, and the second half is exactly $\forall PO$ -freeness.

That is the “yes!” moment for Route 2: **the always-consistent default policy (make everything unconstrained overlap) is also always obligation-free — precisely in this fragment.** The full logic’s nightmare — steering PO edges jointly against watching universals — never engages, because nothing ever watches. The horizontal axis carries no obligations; all real work happens on the vertical skeleton, where the singleton cells make propagation deterministic and blocking sound.

Status, honestly: Route 2 is a *re-derivation* — its ingredients (the normal form, the propagation table, the amalgamation and cross-policy facts) are individually machine-checked, two of them kernel-certified, but the assembly into a standalone tableau-completeness proof is theorem-level prose. It independently confirms the fragment result; it does not independently certify it.

7 The boundary is exactly right — and it is not “horizontal vs. vertical”

A skeptical reader should now ask: is $\forall PO$ -freeness really the boundary, or just a convenient place to stop? Four checks.

The fragment keeps a horizontal universal. Note what did *not* have to go: $\forall DR$. Disjointness is as “horizontal” as overlap, yet $\forall DR.D$ is harmless. The dividing line is not horizontal-vs-vertical but **stable vs. unstable under vertical motion**: DR is transported deterministically by the singleton cells (rains down into parts); PO is transported by no singleton cell in either direction and is never the forced outcome of any single composition. The fragment excises the unstable relation’s universal and nothing else. That the cut lands on a single constructor is a fact about the composition table, not a choice.

The gap on the other side is real. The wandering-witness concept of §5.3 is a *satisfiable* concept just outside the fragment (one $\forall PO$) that provably defeats the constant-interface certificate; the probe wp86 (Part C) confirms the boundary computationally — the PO-incoherent descriptor forces an unsatisfiable interface, the coherent ones never do. And the full logic’s signature satisfiable diagnostic, the strong-EQ PO-loop $C \sqcap \exists PO. \exists PO. C \sqcap \forall \{PO, DR, PP, PPI\}. \neg C$, needs its $\forall PO$ conjunct to force the loop closure — it, too, lives outside. Everything that has ever been *hard* here has a $\forall PO$ in it.

The connection to the open problem is exact. The full-logic keystone F6 asks for a computable live-width bound. Both routes above *prove* one for the fragment (Route 1 explicitly: $K(C_0)$). And the fragment theorem is itself the sharpest available evidence about where the full problem’s difficulty lives: remove *one* constructor — the universal over the one relation the singleton cells don’t control — and the coordination problem that every full-logic attempt has broken on dissolves. ($\forall$ DR also coordinates horizontal edges, but its coordination is *deterministic*, which is exactly why it may stay.) The fragment is thus not a lucky island; it is the complement of the obstruction, and its proof shows *why* the obstruction is where it is.

A corollary: in the fragment, identity can never be forced. The project’s knife’s-edge analysis (the coincidence obstruction of the overview paper) turns on *merge-forcing*: clash out all non-EQ options between two witnesses, and EQ — identity, under strong EQ — is all that remains. Such forced merges are the raw material of grid corners and rigid addressing, the machinery every undecidability attempt needs. In the fragment the trick is inexpressible, by the same escape-valve fact one level up: machine-checked (wp14, part T4), **every composition cell containing EQ also contains PO** — and hence so does every *intersection* of such cells (if EQ survives a joint constraint, every constituent contained EQ, so every constituent contained PO, so PO survives too). Wherever identity is on a pair’s menu, partial overlap is on it as well, and the fragment has no eraser for PO. No $\forall$ PO-free concept ever forces two nodes to be one. So the fragment sits provably on the “loose” side of the knife’s edge: not only does the decidability proof close — the undecidability *machinery* cannot even be assembled.

8 So — was it proved twice, or three times?

Honest ledger. There are **two genuinely different argument shapes** and **three arrivals**, plus one certification pass.

1. **April 2026 — the two-tier quotient** (Claude, reviewed through four revisions by GPT-5.4). Proves decidability for the *PO-coherent* fragment — concepts whose $\forall$ PO obligations, if any, are uniform across phases — with $\forall$ PO-free concepts as the clean special case, and honestly exhibits the PO gap (§5.3) as the boundary. Route 1 above.
2. **July 2026 — the cold F6/W2’ attack** (GPT-5.5, given the open problem cold, asked to attack it). Independently arrived at the $\forall$ PO-free fragment as its “Target B” deliverable, with the $K(C_0)$ bound (papers/po_free_fragment_ALCIRCC5.tex). On inspection its algebra — stabilization, backward forcing, forward absorption, automatic PO-coherence — is the two-tier argument’s, so it counts as an independent *re-derivation* of Route 1, not a new route: the same theorem found again by a different mind from a different starting point, which is worth something evidentially but is not a second proof shape.
3. **July 2026 — the regular-cover pivot** (GPT-5.5, later phase). Re-derives the fragment through the ordered-disjoint normal form and the propagation-closure fact (wp49) — Route 2 above, a genuinely different shape: no chains, no phases, no stabilization; instead a change of data structure that makes PO disappear as data.
4. **2026-07-18 — the certification pass.** After a cold review flagged that the fragment’s original rationale leaned on an overstated “PO is never forced” (the calibration of §4), the

fragment was re-examined: it holds, the argument enforces full composition closure, and the soundness crux — the chain-unfolding lift — was kernel-checked (`P0FreeLift.lean`, zero sorries). Additionally, two *independent* decision procedures (the cover-tree tableau and the quasimodel reasoner) were run on 244 random and curated $\forall$ PO-free concepts with zero mismatches (`wp87`) — a clean cross-validation, because the quasimodel oracle’s one known blind spot needs a $\forall$ PO and so cannot arise in the fragment.

So: “2 (or 3?)” resolves to **two independent proof routes, three independent arrivals at the theorem** — and the fact that a cold attacker, pointed at the open problem with no steer, walked to this exact fragment on its own is perhaps the best evidence that the boundary is natural.

9 What is certified, what is checked, what is argued

Claim	Status
Composition/converse tables; the singleton and PO cells of §4	Kernel-certified (Lean, <code>decide</code>) and derived from set semantics in every probe
RCC5 normal form (network $\Leftrightarrow$ ordered-disjoint), arbitrary domains	Certified , both directions (<code>RCC5NormalForm.lean</code> + canonical representation)
Chain-unfolding lift (finite CC certificate $\Rightarrow$ CC infinite unfolding)	Certified (<code>P0FreeLift.lean</code> , zero sorries)
Certificate-to-model soundness: a valid single-kernel two-tier certificate yields a model of the concept, through the logic (Hintikka + truth lemma on the unfolding)	Certified (<code>P0FreeLift.lean</code> round A, 2026-07-22: <code>twoTier_sound</code> ; witness <code>cinf_satisfiable</code> — the no-finite-model concept $\exists PP.\top \sqcap \forall PP.\exists PP.\top$, satisfiable end-to-end)
Multi-kernel certificates (any family of ascending/descending kernels) and the executable first-order checker (certificates as pure list data; acceptance $\Rightarrow$ model)	Certified (<code>P0FreeLift.lean</code> rounds B–C: <code>multiTier_sound</code> , <code>mtAcceptB_sound</code> ; the ascending+descending two-tower witness accepted by kernel <code>decide</code>)
Propagation closure (no non-PO universal creates a $\forall$ PO obligation)	Machine-checked exhaustively (<code>wp49</code>)
Free amalgamation; all-cross-PO safety; non-uniform cross-policies	Machine-checked exhaustively (<code>wp48</code> , <code>wp82</code> , <code>wp83</code>)
Lift-lemma stress-test incl. multi-path PO forcing; fragment boundary	Machine-checked empirically (<code>wp86</code>)
Two independent reasoners agree on the fragment (244 concepts)	Machine-checked empirically (<code>wp87</code>)
The full fragment theorem ($K(C_0)$ bound, extraction, assembly)	Theorem-level — argued, twice, independently; <i>not</i> end-to-end formalized

Remaining for full certification — the campaign is underway: rounds A–D1 (2026-07-22/23) landed the entire **soundness side plus the decision architecture**: a valid two-tier certificate, with any family of ascending or descending kernels, presented as **pure first-order list data**, is accepted by a computable oracle-free Boolean checker **exactly when valid** (`mt0kB_iff`), acceptance yields a model (kernel-checked end to end; the two-tower witness’s acceptance runs inside the kernel via plain `decide`), and a certified reduction (`decidableSat_of_codes`) turns any candidate-code list plus a completeness premise into Decidable satisfiability. Rounds D2a–c further certified the extraction’s

entire **model-side toolkit**: the stabilization theorem of §5.2 with both forcing corollaries, the infinite pigeonhole (recurrent tails, segment selection), segment coherence (cycling a type-equal chain segment into a kernel is legitimate), witness selection (late picking), and the syntactic $\forall\text{PO}$ -vacuity (`pofree.cl_all`: no $\forall\text{PO}$ ever appears in the fragment’s closure — the escape valve, kernel-checked). Still open: the assembly construction gluing these into an accepted code, and the $K(C_0)$ counting/enumeration (see `LEAN.md` for the recorded design). Until then, the honest label is the project’s standing one: **strongly supported, with the soundness core machine-certified — not certified end-to-end.**

10 The one-paragraph version

$\mathcal{ALCI}_{\text{RCC5}}$ models are complete graphs with a composition law, and they are legitimately infinite, so deciding satisfiability means enumerating and checking *finite certificates* — blueprints whose loops are generators of infinite models — and the whole difficulty is bounding the certificate’s genuinely-free (“live”) content. The composition table transports DR, PP, PPI deterministically along the nesting order (four singleton cells: transitivity, and disjointness raining down into parts) but refuses to transport PO in either direction, and no single composition ever forces PO. Consequently universal obligations over DR/PP/PPI are trackable — they propagate deterministically in a closed finite vocabulary — while $\forall\text{PO}$ obligations must chase neighbors the algebra refuses to pin down; a concrete satisfiable concept with one $\forall\text{PO}$ provably defeats the finite constant-interface certificate via “wandering witnesses” that each overlap a chain only transiently. Ban $\forall\text{PO}$ and both known proof shapes close: the *two-tier quotient* folds each infinite chain into a kernel whose witnesses’ relations stabilize and are all-phase by forced composition (with constant-PO edges algebraically consistent and, now, unchallenged), giving a computable live-width bound and enumerate-and-check decidability; the *ordered-disjoint normal form* — certified — shows a model is nothing but a strict order plus a downward-closed disjointness with PO as the residual non-constraint, so a standard tree tableau tracking only order/disjointness obligations (whose propagation calculus provably never manufactures a $\forall\text{PO}$ obligation) terminates by ordinary blocking, discharging $\exists\text{PO}$ by always-safe free amalgamation. Two shapes, three independent arrivals, the soundness crux kernel-checked in Lean; the fragment is exactly the region where the open keystone F6 — bound the live width — is a theorem instead of a conjecture, and its boundary, one constructor wide, is drawn by the composition table itself.

<i>Companions:</i>	<i>WHY_ITS_HARD.md</i>	(the	full	problem,	non-technical),
<i>papers/two_tier_quotient_ALCIRCC5.tex</i>		(Route	1	in	full),
<i>papers/po_free_fragment_ALCIRCC5.tex</i>		(the	fragment	note),	
<i>papers/rcc5_local_amalgamation_theory.tex</i>		(Route	2’s	local	algebra),
<i>papers/overview_arxiv.tex</i> , §“The win” (the calibrated summary), <i>formal/PDFreeLift.lean</i> , <i>formal/RCC5NormalForm.lean</i> (the certified parts). The Markdown original of this note is <i>papers/WHY_PO_FREE_IS_DECIDABLE.md</i> . Where this document and the formal artifacts disagree, the formal artifacts win.					